Friandises **MATRICIELLES** en **C++**

Peter Philippe

21/04/2026

Ces quelques pages proposent différents calculs numériques (déterminant, norme, inverse, polynôme caractéristique) et des décompositions (**Cholesky**, **LDLT**, **LU**, **QR**, **Diagonalisation**, **Schur**, **Jordan-Chevalley**, **Iwasawa** et **Polaire**), pour des matrices carrées d'ordre 2 à coefficients dans $\mathbb{R}$.

L'ordre étant très petit, les calculs des différents algorithmes proposés seront donc scandaleusement simplifiés dans le code source. La notation utilisée pour les coefficients de la matrice est classique :

$$\mathbf{A} = \begin{pmatrix} a_{1,1} & a_{1,2} \\ a_{2,1} & a_{2,2} \end{pmatrix}$$

Parmi les différentes méthodes exhibées, la décomposition de Jordan peut surprendre par son absence, mais avec un ordre 2, il n'a pas été utile de la mentionner, car cela revient à diagonaliser une matrice. De plus, avec cet ordre « scolaire », la vérification permettant de qualifier qu'une matrice est structurée n'a pas été ajoutée. Ici, le produit matriciel de `operator * (...)` est celui de Strassen (d'autres sont possibles comme celui de Winograd ou de Pan).

(Mieux vaut avoir des bases en algèbre linéaire pour être à l'aise...)

1 Définitions succintes des méthodes de la classe `Matri2x2`

`trace()` La trace est un invariant de similitude par transposition (changement de base), il s'agit simplement d'un scalaire calculé en sommant tous les coefficients de la diagonale de la

matrice $\mathbf{tr}(\mathbf{A}) = \sum_i a_{i,i}$.

`determinant()` Le déterminant est une fonction antisymétrique (ici il se réduit à $|\mathbf{A}| = a_{1,1}a_{2,2} - a_{2,1}a_{1,2}$), lorsqu'il est non nul, l'inversibilité de la matrice sera assurée, elle sera alors qualifiée de « régulière », donc de rang plein, dans le cas contraire, elle sera dite « singulière ». Il intervient notamment dans le calcul du polynôme caractéristique.

`norm()` La norme de Frobenius, encore appelée norme euclidienne, est une fonction retournant une valeur positive, cela permet de mesurer la longueur de la matrice dans son espace et ainsi de pouvoir en comparer la similitude avec une autre matrice, la formule est $\|\mathbf{A}\|_2 = \sqrt{\sum_i \sum_j |a_{i,j}|^2}$. Cette mesure est indispensable en analyse numérique matricielle.

`is_singular()` Une matrice singulière n'est pas inversible, il y a alors au moins une ligne ou une colonne proportionnelle à une autre (famille de vecteurs liés), son noyau est non trivial. Le rang de la matrice est alors strictement inférieur à son ordre.

`is_symmetric()` Une matrice $\mathbf{A}$ est de type symétrique si $\mathbf{A} = \mathbf{A}^{\mathrm{T}}$, ses éléments sont symétriques par rapport à la diagonale ($a_{ij} = a_{ji}$), ici cela se résume à $b = c$. Ces matrices sont toujours diagonalisables et toutes leurs valeurs propres sont réelles.

`is_nilpotent()` Une matrice $\mathbf{A}$ est dite nilpotente si $\mathbf{A}^{k-1} \neq 0$ et $\mathbf{A}^k = 0$ pour un certain $k \in \mathbb{N}^*$; pour un ordre 2, cela équivaut donc à $\mathbf{A}^2 = 0$, sa trace et son déterminant sont nuls.

`is_idempotent()` Une matrice $\mathbf{A}$ idempotente est classée comme telle lorsque $\mathbf{A}^n = \mathbf{A}$ (n correspond à l'ordre de $\mathbf{A}$, donc ici $\mathbf{A}^2 = \mathbf{A}$), ce qui correspond à une projection linéaire ; ses valeurs propres sont 0 ou 1.

`is_involutory()` On dit d'une matrice carrée $\mathbf{A}$ qu'elle est involutive lorsque $\mathbf{AA} = \mathbf{I}$, son déterminant vaut $|\mathbf{A}| = 1$ ou $|\mathbf{A}| = -1$, elle est alors sa propre inverse.

`is_orthogonal()` Une matrice $\mathbf{A}$ est orthogonale si $\mathbf{AA}^{\mathrm{T}} = \mathbf{AA}^{-1} = \mathbf{I}$ (son inverse est donc égal à sa transposée, ce qui est nettement plus rapide à calculer !). Ses colonnes forment une base orthonormée. Par exemple, la matrice de rotation $\mathbf{A} = \left(\begin{smallmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{smallmatrix}\right)$ dans $\mathbb{R}^2$ est un élément du groupe special orthogonal noté $\mathrm{SL}_2(\mathbb{R})$.

`is_upper_unipotent()`
Vérifie que la matrice est triangulaire supérieure avec des 1 sur la diagonale ($a_{1,1} = a_{2,2} = 1$, $a_{2,1} = 0$).

`is_diagonal_positive()`
Teste si la matrice est diagonale ($a_{1,2} = a_{2,1} = 0$) et si ses deux coefficients diagonaux sont strictement positifs.

`transpose()` Retourne la matrice transposée $\mathbf{A}^{\mathrm{T}}$ telle que $a_{i,j} \to a_{j,i}$.

`adjugate()` Retourne la matrice adjointe $\mathbf{A}^*$, utilisée pour le calcul de l'inverse $\mathbf{A}^{-1} = \mathbf{det}(\mathbf{A})^{-1}\mathbf{A}^*$.

`inverse()` Une matrice carrée $\mathbf{A}$ d'ordre n à coefficients dans $\mathbb{K}$ est inversible si et seulement si son déterminant est non nul $|\mathbf{A}| \neq 0$, son rang k est alors nécessairement égal à son ordre n. L'unique solution de $\mathbf{Ax} = \mathbf{0}$ est le vecteur $\mathbf{x} = \mathbf{0}$, cela sous-entend que la famille formée par tous ses vecteurs colonnes est linéairement indépendante (famille libre). L'ensemble des matrices $\mathbf{A}$ inversibles d'ordre n (et diagonalisables) à coefficients dans $\mathbb{R}$ est un groupe linéaire noté $\mathrm{GL}_n(\mathbb{R})$. Dans le cas où la condition de déterminant non nul (ou de rang plein) est satisfaite, la matrice $\mathbf{A}$ d'ordre n

possède une matrice inverse, unique, telle que $\mathbf{A}\mathbf{A}^{-1} = \mathbf{I}$.

`Cholesky()` Décompose une matrice symétrique et définie positive (SDP) en $\mathbf{A} = \mathbf{L}\mathbf{L}^{\mathrm{T}}$, la fonction retourne l'unique matrice triangulaire inférieure $\mathbf{L}$ à éléments diagonaux positifs.

`LDLT()` Lorsque la méthode de Cholesky échoue du fait que la matrice n'est pas (SDP), il faut utiliser la méthode $\mathbf{A} = \mathbf{L}\mathbf{D}\mathbf{L}^{\mathrm{T}}$ avec $\mathbf{L}$ triangulaire inférieure unitaire et $\mathbf{D}$ diagonale, car elle n'utilise plus les racines carrées (ce qui la rend aussi plus stable numériquement). Ici aussi, la matrice $\mathbf{L}$ est unique.

`LU()` Cette factorisation $\mathbf{A} = \mathbf{L}\mathbf{U}$ ne s'applique que si $\mathbf{A}$ est inversible. Cela factorise la matrice en une triangulaire inférieure $\mathbf{L}$ (avec une diagonale unitaire) et une triangulaire supérieure $\mathbf{U}$. Cet algorithme permet de résoudre des systèmes linéaires du type $\mathbf{L}\mathbf{U}\mathbf{x} = \mathbf{b}$ en recherchant $\mathbf{x} = \mathbf{U}^{-1}(\mathbf{L}^{-1}\mathbf{b})$. Cette méthode permet aussi le calcul du déterminant car $|\mathbf{A}| = |\mathbf{U}|$, mais aussi celui de l'inverse de cette façon :

$$\mathbf{A}^{-1} = \mathbf{U}^{-1}\mathbf{L}^{-1}$$

Si le premier pivot $a_{1,1}$ est nul, il faut alors procéder à une permutation des lignes.

`QR()` La décomposition unique $\mathbf{A} = \mathbf{Q}\mathbf{R}$ avec $\mathbf{Q}$ orthogonale et unitaire (ce qui sous-entend ici diagonalisable) et $\mathbf{R}$ triangulaire supérieure telles que $\mathbf{Q}\mathbf{Q}^{\mathrm{T}} = \mathbf{Q}\mathbf{Q}^{-1} = \mathbf{I}$ (donc $\mathbf{Q}^{\mathrm{T}} = \mathbf{Q}^{-1}$). Cette décomposition peut être obtenue par différents procédés (ici, c'est celui de Gram-Schmidt modifié), il s'agit d'un algorithme fondamental en algèbre linéaire numérique. Un cas bien classique de matrice orthogonale est la matrice (non homogène) $\mathbf{R}(\theta)$ du groupe SO(2) (groupe Spécial Orthogonal des matrices d'ordre 2) utilisée pour une rotation d'angle $\theta \in [0,2\pi[$ (lorsque $\theta = 0$ ou $\theta = \pi$, il y a $\mathbf{R}(\theta) = \mathbf{I}$) dans le plan euclidien $\mathbb{R}^2$.

`characteristic_polynomial()`

Retourne le polynôme caractéristique unitaire (dont le degré n correspond à l'ordre de $\mathbf{A}$) tel que :
$\mathcal{P}_{\mathbf{A}}(\lambda) = |\mathbf{A} - \lambda\mathbf{I}| = (-1)^n\lambda^2 - \mathbf{tr}(\mathbf{A})\lambda + \mathbf{det}(\mathbf{A})$. La diagonalisation ou encore le théorème de Cayley-Hamilton utilisent le polynôme caractéristique.

`diagonalization()` Si la matrice est diagonalisable, avec $\mathcal{P}_{\mathbf{A}}$ scindé et à racines simples distinctes dans $\mathbb{C}$, la méthode retourne $(\mathbf{P},\mathbf{D},\mathbf{P}^{-1})$ où $\mathbf{D}$ est diagonale (composée des valeurs propres λ de $\mathbf{A}$) et $\mathbf{P}$ la matrice des vecteurs propres (colonnes) de $\mathbf{A}$.

`schur()` Cette décomposition $\mathbf{A} = \mathbf{SDS}^{\mathrm{T}}$ avec $\mathbf{S}$ unitaire (orthogonale pour les réelles) et $\mathbf{D}$ diagonale ou triangulaire supérieure (ou même être une matrice de rotation dans le cas complexe) ; permet de révéler la structure spectrale de la matrice.

$$\begin{aligned}\mathbf{A} &= \begin{pmatrix} a_{1,1} & a_{1,2} \\ a_{2,1} & a_{2,2} \end{pmatrix} \\ &= \begin{pmatrix} 1 & 0 \\ a_{1,1}^{-1}a_{2,1} & 1 \end{pmatrix} \begin{pmatrix} a_{1,1} & 0 \\ 0 & \mathbf{S} \end{pmatrix} \begin{pmatrix} 1 & a_{1,1}a_{1,2}^{-1} \\ 0 & 1 \end{pmatrix}\end{aligned}$$

avec $\mathbf{S} = a_{2,2} - a_{2,1}a_{1,1}^{-1}a_{1,2}$.

`jordan_chevalley()` Dans un $\mathbb{K}$-espace vectoriel de dimension finie, pour certaines matrices carrées $\mathbf{A} \in \mathcal{M}_n(\mathbb{C})$ de polynôme minimal[1] $\mathcal{P}(\mathbf{A})$ scindé, il existe une unique paire de matrices $\mathbf{D}$ (diagonalisable sur $\mathbb{C}$ uniquement si $\mathcal{P}(\mathbf{A})$ est scindé) et $\mathbf{N}$ (nilpotente) telles que :

$$\mathbf{A} = \mathbf{D} + \mathbf{N} \qquad \mathbf{DN} = \mathbf{ND} \qquad \mathcal{P}(\mathbf{A}) = \mathcal{P}(\mathbf{D})$$

Les matrices d'ordre 2 non diagonalisables vérifient $\mathbf{N} = \mathbf{A} - \mathbf{D}$ et $\mathbf{D} = \lambda\mathbf{I}$ (λ est une valeur propre de $\mathbf{A}$) ainsi

1. Attention, $\mathcal{P}$ est scindé (produits de polynômes de degré 1) si et seulement si le polynôme caractéristique l'est également.

que $\mathbf{N} = \mathbf{A} - \lambda\mathbf{I}$.

`iwasawa()`

Pour des matrices carrées d'ordre 2, cette décomposition d'une matrice $\mathbf{A}$ prend la forme suivante :

$$\mathbf{A} = \mathbf{QDN}$$

avec $\mathbf{Q}$ une matrice orthogonale, $\mathbf{D}$ une matrice diagonale dont les coefficients sont strictement positifs et $\mathbf{N}$ une matrice unipotente unitaire supérieure (donc triangulaire). La matrice $\mathbf{Q}$ s'obtient par le procédé de Gram-Schmidt, ses deux vecteurs colonnes q_1 et q_2 sont :

$$q_1 = \frac{(a_{11},a_{21})^{\mathrm{T}}}{\sqrt{a_{11}^2 + a_{21}^2}} \text{ et } q_2 = \frac{(a_{12},a_{22})^{\mathrm{T}} - (q_1^{\mathrm{T}}(a_{12},a_{22})^{\mathrm{T}})q_1}{\|(a_{12},a_{22})^{\mathrm{T}} - (q_1^{\mathrm{T}}(a_{12},a_{22})^{\mathrm{T}})q_1\|}$$

La diagonale de $\mathbf{D}$ est composée avec les éléments de la diagonale de la matrice $\mathbf{T}$:

$$\mathbf{T} = \mathbf{Q}^{\mathrm{T}}\mathbf{A} = \begin{pmatrix} q_1^{\mathrm{T}}(a_{11},a_{21})^{\mathrm{T}} & q_2^{\mathrm{T}}(a_{12},a_{22})^{\mathrm{T}} \\ q_1^{\mathrm{T}}(a_{11},a_{21})^{\mathrm{T}} & q_2^{\mathrm{T}}(a_{12},a_{22})^{\mathrm{T}} \end{pmatrix}$$

C'est-à-dire :

$$\mathbf{D} = \begin{pmatrix} t_1 & 0 \\ 0 & t_2 \end{pmatrix}$$

Et $\mathbf{N}$ est construite comme ceci :

$$\mathbf{N} = \begin{pmatrix} 1 & t_{1,2}/d_1 \\ 0 & 1 \end{pmatrix}$$

`polar_decomposition()`

Une matrice carrée $\mathbf{A}$ à coefficients dans $\mathbb{R}$ peut être décomposée en un produit de matrices $\mathbf{A} = \mathbf{QS}$ avec $\mathbf{Q}$ une matrice orthogonale et $\mathbf{S}$ une matrice réelle symétrique positive. Lorsque $\mathbf{A}$ est inversible, les matrices $\mathbf{S}$ et $\mathbf{Q}$ sont

uniques. De plus, comme $\mathbf{A} = \mathbf{QS}$ alors $\mathbf{A}^{\mathrm{T}} = \mathbf{SQ}^{\mathrm{T}}$ et donc $\mathbf{A}^{\mathrm{T}}\mathbf{A} = \mathbf{S}^2$. (Ne pas omettre que $\mathbf{A}^{\mathrm{T}}\mathbf{A} \neq \mathbf{AA}^{\mathrm{T}}$.) Le calcul est $\mathbf{P} = \sqrt{\mathbf{A}^{\mathrm{T}}\mathbf{A}}$ puis $\mathbf{Q} = \mathbf{AS}^{-1}$. La méthode `polar_decomposition_general()` retourne $\mathbf{A} = \mathbf{QS}$ avec $\mathbf{Q}$ orthogonale et $\mathbf{S}$ symétrique positive.

2 Implémentation en C++

Comme à mon habitude, ce code de presque 1300 lignes est bien évidemment perfectible (j'aurais quand même pu faire l'effort de rajouter le choix du type dans l'argument au lieu de le laisser au tout début de la fonction `main()`), il ne comporte qu'assez peu de commentaires, sa compilation avec GCC 15.2 s'effectue sans le moindre avertissement. L'utilisation est fort simple puisqu'il ne faut saisir que les 4 coefficients de la matrice (réels ou entiers) séparés par une espace, si aucune valeur n'est saisie, une matrice à coefficients entiers sera générée automatiquement.

```cpp
// g++-mp-15 -march=native -O3 -std=c++23 -lm -Wall -Wpedantic -Wextra -Werror -Wfatal-
    errors matri2x2.cpp -o matri2x2
#include <iostream>
#include <memory>
#include <complex>
#include <exception>
#include <stdexcept>
#include <limits>
#include <optional>
#include <utility>
#include <array>
#include <variant>
#include <tuple>
#include <string>
#include <cmath>
#include <random>
#include <print>
#include <format>
#include <concepts>
#include <string_view>
#include <valarray>

template<typename T>
struct std::formatter<std::complex<T>> {
    constexpr auto parse(std::format_parse_context& context) {
```

```
        return context.begin();
    }

    auto format(const std::complex<T>& c, std::format_context& context) const {
        if (c.imag() == T(0)) {
            return std::format_to(context.out(), "{}", c.real());
        } else if (c.real() == T(0)) {
            return std::format_to(context.out(), "{}i", c.imag());
        } else if (c.imag() < T(0)) {
            return std::format_to(context.out(), "{}{}i", c.real(), c.imag());
        } else {
            return std::format_to(context.out(), "{}+{}i", c.real(), c.imag());
        }
    }
};

template <typename T> concept MatrixType =
    std::is_arithmetic_v<T> ||
    std::is_same_v<T, std::complex<float>> ||
    std::is_same_v<T, std::complex<double>> ||
    std::is_same_v<T, std::complex<long double>>;

template <MatrixType T = float>
class Matri2x2 final {
public:
    using k_type = std::conditional_t<
        std::is_same_v<T, std::complex<float>>, float,
        std::conditional_t<std::is_same_v<T, std::complex<double>>, double,
        std::conditional_t<std::is_same_v<T, std::complex<long double>>, long double, T>>
    >;

    static constexpr k_type _eps = std::numeric_limits<k_type>::epsilon();
    static constexpr size_t N = 2;

    static constexpr bool isComplex =
        std::is_same_v<T, std::complex<float>> ||
        std::is_same_v<T, std::complex<double>> ||
        std::is_same_v<T, std::complex<long double>>;

    using complexType =
        std::conditional_t<std::is_same_v<T, float>, std::complex<float>,
        std::conditional_t<std::is_same_v<T, double>, std::complex<double>,
        std::conditional_t<std::is_same_v<T, long double>, std::complex<long double>, T>
        >>;
```

```

private:
    alignas(64) std::valarray<T> _data;
    mutable std::optional<bool> _singular, _symmetric, _nilpotent, _idempotent, _involutory, _orthogonal, _upper_unipotent, _diagonal_positive;

    constexpr T& a() { return _data[0]; }
    constexpr T& b() { return _data[1]; }
    constexpr T& c() { return _data[2]; }
    constexpr T& d() { return _data[3]; }

    constexpr const T& a() const { return _data[0]; }
    constexpr const T& b() const { return _data[1]; }
    constexpr const T& c() const { return _data[2]; }
    constexpr const T& d() const { return _data[3]; }

public:
    explicit constexpr Matri2x2() noexcept : _data(T(0), 4) {
        a() = T(1);
        d() = T(1);
    }

    explicit constexpr Matri2x2(const T& a_val, const T& b_val, const T& c_val, const T& d_val) noexcept
        : _data(4) {
        a() = a_val;
        b() = b_val;
        c() = c_val;
        d() = d_val;
    }

    explicit constexpr Matri2x2(const std::array<T, 4>& stdarr) noexcept
    : _data(stdarr.data(), 4) {}

    explicit constexpr Matri2x2(std::array<T, 4>&& stdarr) noexcept
    : _data(std::move(stdarr)) {}

    template<MatrixType U>
    explicit constexpr Matri2x2(const Matri2x2<U>& mat) noexcept
        : _data{static_cast<T>(mat[0]), static_cast<T>(mat[1]), static_cast<T>(mat[2]), static_cast<T>(mat[3])} {}

    constexpr Matri2x2(const Matri2x2<T>&) noexcept = default;
    constexpr Matri2x2<T>& operator=(const Matri2x2<T>&) noexcept = default;
```

```cpp

    constexpr Matri2x2(Matri2x2<T>&&) noexcept = default;
    constexpr Matri2x2<T>& operator=(Matri2x2<T>&&) noexcept = default;

    // ~Matri2x2() = default; -> inutile

    constexpr T& operator () (size_t i, size_t j) {
        if (i >= N || j >= N)
            throw std::out_of_range("(i,j) out of matrix!");
        [[assume(i < N && j < N)]];
        return _data[i * N + j];
        // return (i==0 && j==0) ? a_ : (i==0 && j==1) ? b_ : (i==1 && j==0) ? c_ : d_;
    }

    constexpr T operator () (size_t i, size_t j) const {
        if (i >= N || j >= N)
        throw std::out_of_range("(i,j) out of matrix !");
        [[assume(i < N && j < N)]];
        return _data[i * N + j];
        // return (i==0 && j==0) ? a_ : (i==0 && j==1) ? b_ : (i==1 && j==0) ? c_ : d_;
    }

    constexpr const T& operator[](size_t i) const noexcept {
        return _data[i];
    }
    constexpr T& operator[](size_t i) noexcept {
        return _data[i];
    }

    [[nodiscard]] constexpr bool operator == (const Matri2x2<T>& m) const noexcept {
        if constexpr (std::is_integral_v<T>) {
            return a() == m.a() && b() == m.b() && c() == m.c() && d() == m.d();
        } else if constexpr (isComplex) {
            return std::abs(a() - m.a()) <= _eps && std::abs(b() - m.b()) <= _eps &&
                   std::abs(c() - m.c()) <= _eps && std::abs(d() - m.d()) <= _eps;
        } else {
            return std::abs(a() - m.a()) <= _eps && std::abs(b() - m.b()) <= _eps &&
                   std::abs(c() - m.c()) <= _eps && std::abs(d() - m.d()) <= _eps;
        }
    }

    [[nodiscard]] constexpr bool operator != (const Matri2x2<T>& m) const noexcept {
        return !(*this == m);
    }
```

```cpp

    [[nodiscard]] constexpr Matri2x2<T> operator + (const Matri2x2<T>& m) const noexcept {
        Matri2x2<T> result;
        result._data = _data + m._data;
        return result;
    }

    [[nodiscard]] constexpr Matri2x2<T> operator - (const Matri2x2<T>& m) const noexcept {
        Matri2x2<T> result;
        result._data = _data - m._data;
        return result;
    }

    [[nodiscard]] constexpr Matri2x2<T> operator * (const Matri2x2<T>& m) const noexcept {
        if consteval {
            return Matri2x2<T>(
                a() * m.a() + b() * m.c(),
                a() * m.b() + b() * m.d(),
                c() * m.a() + d() * m.c(),
                c() * m.b() + d() * m.d()
            );
        } else {
            if constexpr (!isComplex && !std::is_integral_v<T>) {
                T p1 = (a() + d()) * (m.a() + m.d());
                T p2 = m.a() * (c() + d());
                T p3 = a() * (m.b() - m.d());
                T p4 = d() * (-m.a() + m.c());
                T p5 = m.d() * (a() + b());
                T p6 = (-a() + c()) * (m.a() + m.b());
                T p7 = (b() - d()) * (m.c() + m.d());

                return Matri2x2<T>(
                    p1 + p4 - p5 + p7, p3 + p5,
                    p2 + p4, p1 + p3 - p2 + p6
                );
            } else {
                return Matri2x2<T>(
                    a() * m.a() + b() * m.c(),
                    a() * m.b() + b() * m.d(),
                    c() * m.a() + d() * m.c(),
                    c() * m.b() + d() * m.d()
```

```
                );
            }
        }
    }

    [[nodiscard]] constexpr Matri2x2<T> operator * (const T& scalar) const noexcept {
        Matri2x2<T> result;
        result._data = _data * scalar;
        return result;
    }

    [[nodiscard]] constexpr Matri2x2<T> operator / (const T& scalar) const {
        if (std::abs(scalar) < _eps)
            throw std::runtime_error("Division by zero scalar!");
        [[assume(std::abs(scalar) >= _eps)]];
        return *this * (T(1) / scalar);
    }

    [[nodiscard]] friend constexpr Matri2x2<T> operator * (const T& scalar, const
         Matri2x2<T>& m) noexcept {
        return m * scalar;
    }

    [[nodiscard]] friend constexpr Matri2x2<T> operator / (const T& scalar, const
         Matri2x2<T>& m) {
        if (std::abs(m.determinant()) < m._eps)
            throw std::runtime_error("Matrix is singular!");
        return scalar * m.inverse();
    }

    constexpr Matri2x2<T>& operator+=(const Matri2x2<T>& m) noexcept {
        _data += m._data;
        return *this;
    }

    constexpr Matri2x2<T>& operator-=(const Matri2x2<T>& m) noexcept {
        _data -= m._data;
        return *this;
    }
    constexpr Matri2x2<T>& operator*=(const T& s) noexcept {
        _data *= s;
        return *this;
    }
    constexpr Matri2x2<T>& operator*=(const Matri2x2<T>& m) noexcept {
```

```
        *this = *this * m;
        return *this;
    }
    [[nodiscard]] constexpr T trace() const noexcept {
        return a() + d();
    }

    [[nodiscard]] constexpr T determinant() const noexcept {
        return a() * d() - b() * c();
    }

    [[nodiscard]] constexpr k_type norm() const noexcept {
        if constexpr (isComplex) {
            return std::sqrt(std::norm(a()) + std::norm(b()) + std::norm(c()) + std::norm
                (d()));
        } else {
            k_type sum = 0;
            for (size_t i = 0; i < 4; ++i)
                sum += _data[i] * _data[i];
            return std::sqrt(sum);
        }
    }

    void is_singular() const noexcept requires (!isComplex) {
        _singular = std::abs(this->determinant()) <= _eps;
    }

    void is_symmetric() const noexcept requires (!isComplex) {
        _symmetric = std::abs(b() - c()) <= _eps;
    }

    void is_nilpotent() const noexcept requires (!isComplex) {
        auto mat = (*this) * (*this);
        _nilpotent = std::abs(mat.a()) <= _eps &&
                     std::abs(mat.b()) <= _eps &&
                     std::abs(mat.c()) <= _eps &&
                     std::abs(mat.d()) <= _eps;
    }

    void is_idempotent() const noexcept requires (!isComplex) {
        auto mat = (*this) * (*this);
        _idempotent = std::abs(mat.a() - a()) <= _eps &&
                      std::abs(mat.b() - b()) <= _eps &&
                      std::abs(mat.c() - c()) <= _eps &&
```

```
                    std::abs(mat.d() - d()) <= _eps;
    }

    void is_involutory() const noexcept requires (!isComplex) {
        auto mat = (*this) * (*this);
        _involutory = std::abs(mat.a() - T(1)) <= _eps &&
                      std::abs(mat.b()) <= _eps &&
                      std::abs(mat.c()) <= _eps &&
                      std::abs(mat.d() - T(1)) <= _eps;
    }

    void is_orthogonal() const noexcept requires (!isComplex) {
        auto mt = this->transpose();
        auto mmt = mt * (*this);
        _orthogonal = std::abs(mmt.a() - T(1)) <= _eps &&
                      std::abs(mmt.b()) <= _eps &&
                      std::abs(mmt.c()) <= _eps &&
                      std::abs(mmt.d() - T(1)) <= _eps;
    }

    void is_upper_unipotent() const noexcept requires (!isComplex) {
        _upper_unipotent = std::abs(a() - T(1)) <= _eps &&
                           std::abs(d() - T(1)) <= _eps &&
                           std::abs(c()) <= _eps;
    }

    void is_diagonal_positive() const noexcept requires (!isComplex) {
        _diagonal_positive = std::abs(b()) <= _eps &&
                             std::abs(c()) <= _eps &&
                             a() > 0 && d() > 0;
    }

    void verifications() const noexcept requires (!isComplex) {
        is_singular();
        is_symmetric();
        is_nilpotent();
        is_idempotent();
        is_involutory();
        is_orthogonal();
        is_upper_unipotent();
        is_diagonal_positive();
    }

```

```
    [[nodiscard]] constexpr bool is_singular_acces() const noexcept requires (!isComplex
        ) {
        return _singular.value_or(false);
    }

    [[nodiscard]] constexpr bool is_symmetric_acces() const noexcept requires (!
        isComplex) {
        return _symmetric.value_or(false);
    }

    [[nodiscard]] constexpr bool is_nilpotent_acces() const noexcept requires (!
        isComplex) {
        return _nilpotent.value_or(false);
    }

    [[nodiscard]] constexpr bool is_idempotent_acces() const noexcept requires (!
        isComplex) {
        return _idempotent.value_or(false);
    }

    [[nodiscard]] constexpr bool is_involutory_acces() const noexcept requires (!
        isComplex) {
        return _involutory.value_or(false);
    }

    [[nodiscard]] constexpr bool is_orthogonal_acces() const noexcept requires (!
        isComplex) {
        return _orthogonal.value_or(false);
    }

    [[nodiscard]] constexpr bool is_upper_unipotent_acces() const noexcept requires (!
        isComplex) {
        return _upper_unipotent.value_or(false);
    }

    [[nodiscard]] constexpr bool is_diagonal_positive_acces() const noexcept requires (!
        isComplex) {
        return _diagonal_positive.value_or(false);
    }

    [[nodiscard]] constexpr Matri2x2<T> transpose() const noexcept {
        return Matri2x2(a(), c(), b(), d());
    }

```

```
    [[nodiscard]] constexpr Matri2x2<T> adjugate() const noexcept {
        return Matri2x2(d(), -b(), -c(), a());
    }

    [[nodiscard]] Matri2x2<T> inverse() const {
        T det = determinant();

        k_type absDet;
        if constexpr (isComplex) {
            absDet = std::abs(det);
        } else {
            absDet = std::abs(det);
        }

        if (absDet <= _eps)
            throw std::runtime_error("Singular matrix !");
        T invDet = T(1) / det;

        return Matri2x2(d() * invDet, -b() * invDet, -c() * invDet, a() * invDet);
    }

    [[nodiscard]] auto Cholesky() const -> Matri2x2<T> requires (!isComplex) {
        if (!_symmetric.value_or(false))
            throw std::runtime_error("Unsymmetric matrix !");

        if (a() <= 0 || (a() * d() - b() * c()) <= 0)
            throw std::runtime_error("Not SPD matrix.");

        T l11 = std::sqrt(a());
        [[assume(l11 > T(0))]];
        T l21 = c() / l11;
        T l22 = std::sqrt(d() - l21 * l21);

        return Matri2x2<T>(l11, 0, l21, l22);
    }

    [[nodiscard]] auto LDLT() const -> std::tuple<Matri2x2<T>, Matri2x2<T>, Matri2x2<T>>
         requires (!isComplex) {
        if (!_symmetric.value_or(false))
            throw std::runtime_error("Unsymmetric matrix!");

        if (std::abs(a()) < _eps)
            throw std::runtime_error("Pivot too small or equal to zero!");

```

```
        T d1 = a();
        T l21 = b() / a();
        T d2 = d() - l21 * b();

        Matri2x2<T> L(T(1), T(0), l21, T(1));
        Matri2x2<T> D(d1, T(0), T(0), d2);
        Matri2x2<T> LT = L.transpose();

        return {L, D, LT};
    }

    [[nodiscard]] auto LU() const -> std::pair<Matri2x2<T>, Matri2x2<T>> {
        if (std::abs(a()) < _eps)
            throw std::runtime_error("Pivot too small or equal to zero!");
        [[assume(std::abs(a()) >= _eps)]];
        T l21 = c() / a();
        Matri2x2<T> L(T(1), T(0), l21, T(1));
        Matri2x2<T> U(a(), b(), T(0), d() - l21 * b());
        return {L, U};
    }

    [[nodiscard]] auto QR() const -> std::pair<Matri2x2<T>, Matri2x2<T>> requires (!
        isComplex) {
        T normeCol1 = std::hypot(a(), c());
        if (normeCol1 < _eps) {
            std::cerr << "Warning: First column is near zero, using default orthonormal
                basis." << std::endl;
            std::exit(EXIT_FAILURE);
        } else {
            [[assume(normeCol1 > _eps)]];
        }
        T q11 = (normeCol1 > _eps) ? a() / normeCol1 : T(0);
        T q21 = (normeCol1 > _eps) ? c() / normeCol1 : T(0);

        T dp = q11 * b() + q21 * d();
        T col2X = b() - dp * q11;
        T col2Y = d() - dp * q21;

        T normeCol2 = std::hypot(col2X, col2Y);
        T q12 = (normeCol2 > _eps) ? col2X / normeCol2 : T(0);
        T q22 = (normeCol2 > _eps) ? col2Y / normeCol2 : T(0);

        Matri2x2<T> Q(q11, q12, q21, q22);
        Matri2x2<T> R = Q.transpose() * (*this);
```

```

        return {Q, R};
    }

    [[nodiscard]] std::string characteristic_polynomial() const noexcept requires (!isComplex) {
        T det = determinant();
        T tr = trace();

        return std::format("P(x) = x²{}{}{}",
            tr > 0 ? std::format(" - {}x", tr) : (tr < 0 ? std::format(" + {}x", -tr) : ""),
            det > 0 ? std::format(" + {}", det) : (det < 0 ? std::format(" - {}", -det) : ""),
            (tr == 0 && det == 0) ? "0" : "");
    }

    [[nodiscard]] std::variant<
        std::pair<T, T>,
        std::pair<std::complex<T>, std::complex<T>>
    > eigenvalues() const noexcept requires (!isComplex) {
        const T tr = trace();
        const T det = determinant();
        const T disc = tr * tr - 4.0 * det;

        if (disc >= 0) {
            T sqrtDisc = std::sqrt(disc);
            T lambda1 = (tr - sqrtDisc) * 0.5;
            T lambda2 = (tr + sqrtDisc) * 0.5;
            return std::pair<T, T>(lambda1, lambda2);
        } else {
            T realPart = tr * 0.5;
            T imagPart = std::sqrt(-disc) * 0.5;
            return std::pair<std::complex<T>, std::complex<T>>(
                std::complex<T>(realPart, imagPart),
                std::complex<T>(realPart, -imagPart)
            );
        }
    }

    [[nodiscard]] std::variant<
        std::pair<std::array<T, 2>, std::array<T, 2>>,
        std::pair<std::array<std::complex<T>, 2>, std::array<std::complex<T>, 2>>
    > eigenvectors() const noexcept requires (!isComplex) {
```

```
        auto eigval = eigenvalues();

        if (std::holds_alternative<std::pair<T, T>>(eigval)) {
            auto [lambda1, lambda2] = std::get<0>(eigval);
            std::array<T, 2> v1, v2;

            if (std::abs(c()) > _eps) {
                v1[0] = d() - lambda1;
                v1[1] = -c();
            } else if (std::abs(b()) > _eps) {
                v1[0] = -b();
                v1[1] = a() - lambda1;
            } else {
                v1 = {1, 0};
            }

            T norm1 = std::hypot(v1[0], v1[1]);
            if (norm1 > _eps) {
                v1[0] /= norm1;
                v1[1] /= norm1;
            }

            if (std::abs(c()) > _eps) {
                v2[0] = d() - lambda2;
                v2[1] = -c();
            } else if (std::abs(b()) > _eps) {
                v2[0] = -b();
                v2[1] = a() - lambda2;
            } else {
                v2 = {0, 1};
            }

            T norm2 = std::hypot(v2[0], v2[1]);
            if (norm2 > _eps) {
                v2[0] /= norm2;
                v2[1] /= norm2;
            }

            return std::pair<std::array<T, 2>, std::array<T, 2>>(v1, v2);
        } else {
            auto [lambda1, lambda2] = std::get<1>(eigval);
            std::array<std::complex<T>, 2> v1;

            if (std::abs(c()) > _eps) {
```

```
            v1[0] = std::complex<T>(d(), 0) - lambda1;
            v1[1] = std::complex<T>(-c(), 0);
        } else if (std::abs(b()) > _eps) {
            v1[0] = std::complex<T>(-b(), 0);
            v1[1] = std::complex<T>(a(), 0) - lambda1;
        } else {
            v1 = {std::complex<T>(1, 0), std::complex<T>(0, 0)};
        }

        T norm = std::sqrt(std::norm(v1[0]) + std::norm(v1[1]));
        if (norm > _eps) {
            v1[0] /= norm;
            v1[1] /= norm;
        }

        std::array<std::complex<T>, 2> v2 = {std::conj(v1[0]), std::conj(v1[1])};

        return std::pair<std::array<std::complex<T>, 2>, std::array<std::complex<T>,
            2>>(v1, v2);
    }
}

[[nodiscard]] auto diagonalization() const -> std::variant<
    std::tuple<Matri2x2<T>, Matri2x2<T>, Matri2x2<T>>,
    std::tuple<Matri2x2<complexType>, Matri2x2<complexType>, Matri2x2<complexType>>
> requires (!isComplex) {
    auto eigval = eigenvalues();
    auto eigvec = eigenvectors();

    if (std::holds_alternative<std::pair<T, T>>(eigval)) {
        auto [lambda1, lambda2] = std::get<0>(eigval);
        auto [v1, v2] = std::get<0>(eigvec);

        T det_P = v1[0] * v2[1] - v1[1] * v2[0];
        if (std::abs(det_P) < _eps)
            throw std::runtime_error("-> Colinear eigenvectors!");

        Matri2x2<T> P(v1[0], v2[0], v1[1], v2[1]);
        Matri2x2<T> D(lambda1, 0, 0, lambda2);
        Matri2x2<T> invP = P.inverse();

        return std::tuple<Matri2x2<T>, Matri2x2<T>, Matri2x2<T>>{P, D, invP};
    }
    else {
```

```
            auto [lambda1, lambda2] = std::get<1>(eigval);
            auto [v1, v2] = std::get<1>(eigvec);

            using cplx = complexType;

            Matri2x2<cplx> P(v1[0], v2[0], v1[1], v2[1]);
            Matri2x2<cplx> D(lambda1, cplx(0,0), cplx(0,0), lambda2);

            cplx detP = v1[0] * v2[1] - v1[1] * v2[0];
            if (std::abs(detP) < _eps)
               throw std::runtime_error("-> Colinear eigenvectors!");

            Matri2x2<cplx> invP = P.inverse();

            return std::tuple<Matri2x2<cplx>, Matri2x2<cplx>, Matri2x2<cplx>>{P, D, invP
                };
        }
    }

    [[nodiscard]] auto Schur() const -> std::pair<Matri2x2<T>, Matri2x2<T>> requires (!
        isComplex) {
        auto eigval = eigenvalues();

        if (std::holds_alternative<std::pair<std::complex<T>, std::complex<T>>>(eigval))
            {
            auto [lambda1, lambda2] = std::get<1>(eigval);
            T alpha = lambda1.real();
            T beta = lambda1.imag();

            std::complex<T> a11 = std::complex<T>(a(), 0) - lambda1;
            std::complex<T> a12 = std::complex<T>(b(), 0);
            std::complex<T> a21 = std::complex<T>(c(), 0);
            std::complex<T> a22 = std::complex<T>(d(), 0) - lambda1;

            std::complex<T> v1, v2;
            if (std::abs(a12) > _eps) {
                v2 = std::complex<T>(1, 0);
                v1 = -a12 / a11;
            } else if (std::abs(a21) > _eps) {
                v1 = std::complex<T>(1, 0);
                v2 = -a21 / a22;
            } else {
                v1 = std::complex<T>(1, 0);
                v2 = std::complex<T>(0, 0);
```

```
        }

        T norm = std::sqrt(std::norm(v1) + std::norm(v2));
        if (norm > _eps) {
            v1 /= norm;
            v2 /= norm;
        }

        T q11 = v1.real(), q21 = v2.real();
        T q12 = v1.imag(), q22 = v2.imag();

        T normCol1 = std::hypot(q11, q21);
        T normCol2 = std::hypot(q12, q22);

        if (normCol1 > _eps) { q11 /= normCol1; q21 /= normCol1; }
        if (normCol2 > _eps) { q12 /= normCol2; q22 /= normCol2; }

        T dot = q11 * q12 + q21 * q22;
        if (std::abs(dot) > _eps) {
            q12 -= dot * q11;
            q22 -= dot * q21;
            T norm_new = std::hypot(q12, q22);
            if (norm_new > _eps) { q12 /= norm_new; q22 /= norm_new; }
        }

        Matri2x2<T> Q(q11, q12, q21, q22);
        Matri2x2<T> T_mat(alpha, beta, -beta, alpha);

        Matri2x2<T> test = Q * T_mat * Q.transpose();
        T error = std::abs(test(0,0) - (*this)(0,0)) + std::abs(test(0,1) - (*this)(0,1)) +
                std::abs(test(1,0) - (*this)(1,0)) + std::abs(test(1,1) - (*this)(1,1));

        if (error > _eps)
            T_mat = Q.transpose() * (*this) * Q;

        return {Q, T_mat};
    }

    auto [lambda1, lambda2] = std::get<0>(eigval);

    bool swapNeeded = false;
    if (std::abs(lambda1) > std::abs(lambda2)) {
```

```
            std::swap(lambda1, lambda2);
            swapNeeded = true;
        }

        T a11 = a() - lambda1;
        T a12 = b();
        T a21 = c();
        T a22 = d() - lambda1;

        T vX, vY;
        if (std::abs(a12) > std::abs(a22) && std::abs(a12) > std::abs(a21)) {
            vY = 1;
            vX = -a12 / (a11 + _eps);
        } else if (std::abs(a21) > std::abs(a11) && std::abs(a21) > std::abs(a12)) {
            vX = 1;
            vY = -a21 / (a22 + _eps);
        } else if (std::abs(a11) > _eps) {
            vY = 1;
            vX = -a12 / a11;
        } else if (std::abs(a22) > _eps) {
            vX = 1;
            vY = -a21 / a22;
        } else {
            vX = 1;
            vY = 0;
        }

        T norm = std::hypot(vX, vY);
        if (norm > _eps) {
            vX /= norm;
            vY /= norm;
        }

        T uX = -vY;
        T uY = vX;

        Matri2x2<T> Q = swapNeeded ? Matri2x2<T>(uX, vX, uY, vY) : Matri2x2<T>(vX, uX, vY
            , uY);
        Matri2x2<T> T_mat = Q.transpose() * (*this) * Q;
        if (std::abs(T_mat(1,0)) < _eps)
            T_mat(1,0) = T(0);

        return {Q, T_mat};
    }
```

```cpp

    [[nodiscard]] auto jordan_chevalley() const -> std::pair<Matri2x2<T>, Matri2x2<T>>
        requires (!isComplex) {
        const T tr = trace();
        const T det = determinant();
        const T disc = tr * tr - T(4) * det;

        if (disc >= 0) {
            // Real eigenvalues
            const T sqrtDisc = std::sqrt(disc);
            const T lambda1 = (tr - sqrtDisc) / T(2);
            const T lambda2 = (tr + sqrtDisc) / T(2);

            if (std::abs(lambda1 - lambda2) > _eps) {
                // Distinct eigenvalues => diagonalizable
                return {*this, Matri2x2<T>(T(0), T(0), T(0), T(0))};
            } else {
                // Double eigenvalue
                const T lambda = lambda1;
                const Matri2x2<T> d(lambda, T(0), T(0), lambda);
                const Matri2x2<T> n = (*this) - d;
                // Check if N² = 0
                const auto nn = n * n;
                const T nilpotence = std::abs(nn[0]) + std::abs(nn[1]) +
                                     std::abs(nn[2]) + std::abs(nn[3]);
                if (nilpotence < _eps && n.norm() > _eps) {
                    // Non-diagonalizable
                    return {d, n};
                } else {
                    // Diagonalizable (A = lambda I)
                    return {*this, Matri2x2<T>(T(0), T(0), T(0), T(0))};
                }
            }
        } else {
            return {*this, Matri2x2<T>(T(0), T(0), T(0), T(0))};
        }
    }
    [[nodiscard]] auto iwasawa() const -> std::tuple<Matri2x2<T>, Matri2x2<T>, Matri2x2<
        T>> requires (!isComplex) {
        if (std::abs(determinant()) < _eps)
            throw std::runtime_error("Singular matrix!");
        T normCol1 = std::hypot(a(), c());
        if (normCol1 < _eps)
            throw std::runtime_error("First column is zero!");
```

```
        [[assume(normCol1 >= _eps)]];

        T k11 = a() / normCol1;
        T k21 = c() / normCol1;
        T dot = k11 * b() + k21 * d();
        T projX = dot * k11;
        T projY = dot * k21;
        T orthoX = b() - projX;
        T orthoY = d() - projY;

        T normOrtho = std::hypot(orthoX, orthoY);
        if (normOrtho < _eps) {
            k11 = 1.0; k21 = 0.0;
            orthoX = 0.0; orthoY = 1.0;
            normOrtho = 1.0;
        }

        T k12 = orthoX / normOrtho;
        T k22 = orthoY / normOrtho;

        // det(K) = -1 or 1
        Matri2x2<T> K(k11, k12, k21, k22);

        // If det(K) = -1, multiply the second column by -1
        T detK = K.determinant();
        if (detK < 0) {
            k12 = -k12;
            k22 = -k22;
            K = Matri2x2<T>(k11, k12, k21, k22);
        }

        // Étape 2: Compute A' = K^T * A
        Matri2x2<T> Aprime = K.transpose() * (*this);

        Aprime(1, 0) = T(0);

        // Étape 3: Extract A_diag and N de A'
        // A' = [[d1, x], [0, d2]] = [[d1, 0], [0, d2]] * [[1, n], [0, 1]]
        // where n = x / d1

        T d1 = Aprime(0, 0);
        T d2 = Aprime(1, 1);
        T x = Aprime(0, 1);

```

```
        if (std::abs(d1) < _eps || std::abs(d2) < _eps)
            throw std::runtime_error("Diagonal elements too small!");

        // Assurer d1, d2 > 0
        if (d1 < 0) {
            d1 = -d1;
            k11 = -k11;
            k21 = -k21;
            K = Matri2x2<T>(k11, k12, k21, k22);
        }

        if (d2 < 0) {
            d2 = -d2;
            k12 = -k12;
            k22 = -k22;
            K = Matri2x2<T>(k11, k12, k21, k22);
        }

        Matri2x2<T> diagA(d1, T(0), T(0), d2);
        T n = x / d1;
        Matri2x2<T> N(T(1), n, T(0), T(1));

        return {K, diagA, N};
    }

    [[nodiscard]] auto polar_decomposition() const -> std::pair<Matri2x2<T>, Matri2x2<T
        >>
        requires (!isComplex) {
        if (!_symmetric.value_or(false))
            throw std::runtime_error("Unsymmetric matrix!");

        if (a() <= 0 || determinant() <= 0)
            throw std::runtime_error("Matrix must be positive definite!");

        T traceA = trace();
        T detA = determinant();
        T sqrtDet = std::sqrt(detA);
        T s = std::sqrt(traceA + T(2) * sqrtDet);

        if (std::abs(s) < _eps)
            throw std::runtime_error("Polar decomposition: singular case!");

        Matri2x2<T> sqrtA;

```

```
        if (std::abs(s) > _eps) {
            T t = T(1) / s;
            sqrtA = (*this) + Matri2x2<T>(sqrtDet, T(0), T(0), sqrtDet);
            sqrtA *= t;
        } else {
            T lambda = std::sqrt(a());
            sqrtA = Matri2x2<T>(lambda, T(0), T(0), lambda);
        }

        Matri2x2<T> Q = Matri2x2<T>(T(1), T(0), T(0), T(1));

        return {Q, sqrtA};
    }

    [[nodiscard]] auto polar_decomposition_general() const
        -> std::pair<Matri2x2<T>, Matri2x2<T>> requires (!isComplex)
    {
        Matri2x2<T> mtm = this->transpose() * (*this);
        T a = mtm(0,0), b = mtm(0,1), d = mtm(1,1);

        T traceMtm = a + d;
        T detMtm = a * d - b * b;
        T disc = traceMtm * traceMtm - T(4) * detMtm;

        if (disc < 0)
            throw std::runtime_error("Polar decomposition: mtm has complex eigenvalues!")
                ;
        T sqrt_disc = std::sqrt(disc);
        T lambda1 = (traceMtm - sqrt_disc) / T(2);
        T lambda2 = (traceMtm + sqrt_disc) / T(2);

        if (lambda1 < 0 || lambda2 < 0)
            throw std::runtime_error("Polar decomposition: mtm not positive semidefinite!
                ");

        T sqrt_lambda1 = std::sqrt(lambda1);
        T sqrt_lambda2 = std::sqrt(lambda2);

        Matri2x2<T> P;

        if (std::abs(b) < _eps) {
            // Diagonal case
            P = Matri2x2<T>(std::sqrt(a), T(0), T(0), std::sqrt(d));
        } else {
```

```
            // Eigenvector for lambda1
            T v1x, v1y;
            if (std::abs(a - lambda1) > _eps) {
                v1x = T(1);
                v1y = -(a - lambda1) / b;
            } else {
                v1y = T(1);
                v1x = -b / (d - lambda1);
            }
            T norm1 = std::hypot(v1x, v1y);
            v1x /= norm1;
            v1y /= norm1;

            // Eigenvector for lambda2
            T v2x = -v1y;
            T v2y = v1x;

            // Rotation matrix R = [v1 v2]
            Matri2x2<T> R(v1x, v2x, v1y, v2y);
            Matri2x2<T> D_sqrt(sqrt_lambda1, T(0), T(0), sqrt_lambda2);

            // P = R * D_sqrt * R^T
            P = R * D_sqrt * R.transpose();
        }

        // U = M * P^{-1}
        Matri2x2<T> P_inv = P.inverse();
        Matri2x2<T> U = (*this) * P_inv;

        // Optional: force U to be exactly orthogonal (clean numerical noise)
        // U = (U + U.inverse().transpose()) / T(2); // polar projection

        return {U, P};
    }
    [[nodiscard]] std::string display(int precision) const {
        if constexpr (isComplex) {
            auto format_complex = [precision](const T& c) -> std::string {
                k_type re = c.real();
                k_type im = c.imag();

                if (std::abs(im) < _eps) {
                    return std::format("{:.{}f}", re, precision);
                } else if (std::abs(re) < _eps) {
                    return std::format("{:.{}f}i", im, precision);
```

```
                } else if (im < 0) {
                    return std::format("{:.{}f}{:.{}f}i", re, precision, im, precision);
                } else {
                    return std::format("{:.{}f}+{:.{}f}i", re, precision, im, precision);
                }
            };

            return std::format("|{} {}|\n|{} {}|",
                format_complex(a()), format_complex(b()),
                format_complex(c()), format_complex(d()));
        } else {
            return std::format("|{:.{}f} {:.{}f}|\n|{:.{}f} {:.{}f}|",
                a(), precision, b(), precision,
                c(), precision, d(), precision);
        }
    }

    friend std::ostream& operator<<(std::ostream& os, const Matri2x2<T>& m) {
        return os << m.display();
    }
};

// template<typename T>
// bool commutes(const Matri2x2<T>& A, const Matri2x2<T>& B) {
//     auto AB = A * B;
//     auto BA = B * A;
//     return (AB - BA).norm() < Matri2x2<T>::_eps;
// }

template<typename T> concept FloatingPrecision = std::same_as<T, float> || std::same_as<T, double> || std::same_as<T, long double>;

template<FloatingPrecision T> int get_precision() {
    if constexpr (std::same_as<T, float>) {
        int p = 6;
        std::println("Using single precision (float {})", p);
        return p;
    } else if constexpr (std::same_as<T, double>) {
        int p = 12;
        std::println("Using double precision (double {})", p);
        return p;
    } else if constexpr (std::same_as<T, long double>) {
        int p = 15;
        std::println("Using extended precision (long double {})", p);
```

```
        return p;
    } else {
        return 0;
    }
}

int main(int argc, char **argv) {
    using T = float; // float or double or long double
    T a = {}, b = {}, c = {}, d = {};

    int precision = get_precision<T>();
    if (precision == 0) {
        std::println(stderr, "Unsupported type for precision info.");
        std::exit(EXIT_FAILURE);
    }

    if (argc == 5) {
        auto parse = [](const char* str) -> T {
            char* end = nullptr;
            T coefs = std::strtof(str, &end);
            if (end == str || *end != '\0') {
                throw std::runtime_error(std::format("Invalid number: '{}'", str));
            }
            return coefs;
        };
        try {
            a = parse(argv[1]);
            b = parse(argv[2]);
            c = parse(argv[3]);
            d = parse(argv[4]);
        } catch (const std::runtime_error& e) {
            std::println(stderr, "Error parsing arguments: {}", e.what());
            std::exit(EXIT_FAILURE);
        }
    } else {
        std::random_device rnd;
        std::mt19937 gen(rnd());
        std::uniform_int_distribution<int> dist(-10, 10);
        a = dist(gen);
        b = dist(gen);
        c = dist(gen);
        d = dist(gen);
    }

```

```
    auto matrixPtr = std::make_unique<Matri2x2<T>>(a, b, c, d);
    Matri2x2<T>& matrix = *matrixPtr;

    if constexpr (!Matri2x2<T>::isComplex) {
        matrix.verifications();
    }

    std::println("\nMatrix:\n{}\n", matrix.display(precision));

    if constexpr (!Matri2x2<T>::isComplex) {
        std::println("Is singular ? {}", matrix.is_singular_acces() ? "Yes" : "No");
        std::println("Is symmetric ? {}", matrix.is_symmetric_acces() ? "Yes" : "No");
        std::println("Is nilpotent (A² = 0) ? {}", matrix.is_nilpotent_acces() ? "Yes" : "No");
        std::println("Is idempotent (A² = A) ? {}", matrix.is_idempotent_acces() ? "Yes" : "No");
        std::println("Is involutory (A² = I) ? {}", matrix.is_involutory_acces() ? "Yes" : "No");
        std::println("Is orthogonal (AA^T = I) ? {}\n", matrix.is_orthogonal_acces() ? "Yes" : "No");
    } else {
        std::unreachable();
    }

    std::println("Trace: {:.{}f}", matrix.trace(), precision);
    std::println("Determinant: {:.{}f}", matrix.determinant(), precision);
    std::println("Frobenius norm: {:.{}f}", matrix.norm(), precision);
    std::println();
    std::println("Adjugate:\n{}\n", matrix.adjugate().display(precision));

    std::string msgNonInv = "Singular matrix!\n)";
    if constexpr (!Matri2x2<T>::isComplex) {
        if (matrix.is_singular_acces())
            std::println("", msgNonInv);
    } else {
        if (std::abs(matrix.determinant()) <= Matri2x2<T>::_eps)
            std::println("", msgNonInv);
    }

    try {
        if (!matrix.is_singular_acces()) {
            std::println("Inverse:\n{}\n", matrix.inverse().display(precision));
            std::println("AA^(-1):\n{}\n", (matrix.inverse() * matrix).display(precision));
```

```
        }
    } catch (const std::runtime_error& e) {
        std::println(stderr, "Erreur: {}", e.what());
    }

    if constexpr (!Matri2x2<T>::isComplex) {
        if (matrix.is_symmetric_acces()) {
            std::println("Cholesky decomposition (symmetric matrix):");
            std::println("-----------------------------------------");
            try {
                auto C = matrix.Cholesky();
                std::println("L:\n{}", C.display(precision));
                std::println("LL^T:\n{}\n", (C * C.transpose()).display(precision));
            } catch (const std::runtime_error& e) {
                std::println(stderr, "Cholesky: {}", e.what());
                std::println("Attempt with LDLT:");
                try {
                    auto [L, D, LT] = matrix.LDLT();
                    std::println("L:\n{}", L.display(precision));
                    std::println("D:\n{}", D.display(precision));
                    std::println("L^T:\n{}", LT.display(precision));
                    std::println("LDL^T:\n{}\n", (L * D * LT).display(precision));
                } catch (const std::runtime_error& e2) {
                    std::println(stderr, "LDLT: {}\n", e2.what());
                }
            }
        } else {
            std::println("LU decomposition (unsymmetric matrix):");
            std::println("--------------------------------------");
            try {
                auto [L, U] = matrix.LU();
                std::println("L:\n{}", L.display(precision));
                std::println("U:\n{}", U.display(precision));
                std::println("LU:\n{}\n", (L * U).display(precision));
            } catch (const std::runtime_error& e) {
                std::println(stderr, "LU: {}\n", e.what());
            }
        }

        std::println("QR decomposition:");
        std::println("-----------------");
        try {
            auto [Q, R] = matrix.QR();
            std::println("Q:\n{}", Q.display(precision));
```

```
            std::println("R:\n{}", R.display(precision));
            std::println("QR:\n{}\n", (Q * R).display(precision));
        } catch (const std::runtime_error& e) {
            std::println(stderr, "QR: {}\n", e.what());
        }

        std::println("Characteristic polynomial:");
        std::println("--------------------------");
        std::println("{}", matrix.characteristic_polynomial());

        auto valp = matrix.eigenvalues();
        if (std::holds_alternative<std::pair<T, T>>(valp)) {
            auto [l1, l2] = std::get<0>(valp);
            std::println("Real eigenvalues (roots): {:.{}f}, {:.{}f}\n", l1, precision, l2, precision);
        } else {
            auto [l1, l2] = std::get<1>(valp);
            std::println("Complex eigenvalues: {}, {}\n", l1, l2);
        }

        auto vecp = matrix.eigenvectors();
        if (std::holds_alternative<std::pair<std::array<T, 2>, std::array<T, 2>>>(vecp)) {
            auto [v1, v2] = std::get<0>(vecp);
            std::println("Real eigenvectors:");
            std::println(" v1 = [{:.{}f}, {:.{}f}]", v1[0], precision, v1[1], precision);
            std::println(" v2 = [{:.{}f}, {:.{}f}]\n", v2[0], precision, v2[1], precision);
        } else {
            auto [v1, v2] = std::get<1>(vecp);
            std::println("Complex eigenvectors:");
            std::println(" v1 = [{}, {}]", v1[0], v1[1]);
            std::println(" v2 = [{}, {}]\n", v2[0], v2[1]);
        }

        std::println("\nDiagonalization:");
        std::println("----------------");
        try {
            auto result = matrix.diagonalization();

            if (std::holds_alternative<std::tuple<Matri2x2<T>, Matri2x2<T>, Matri2x2<T>>>(result)) {
                auto [P, D, invP] = std::get<0>(result);
                std::println("Passage matrix P:\n{}\n", P.display(precision));
```

```
                std::println("Diagonal matrix D:\n{}\n", D.display(precision));
                std::println("P^(-1):\n{}\n", invP.display(precision));
                std::println("A = PDP^(-1):\n{}\n", (P * D * invP).display(precision));
            } else {
                auto [P, D, invP] = std::get<1>(result);
                std::println("Passage matrix P (complex):\n{}\n", P.display(precision));
                std::println("Diagonal matrix D (complex):\n{}\n", D.display(precision));
                std::println("P^(-1) (complex):\n{}\n", invP.display(precision));
                std::println("A = PDP^(-1):\n{}", (P * D * invP).display(precision));
            }
        } catch (const std::runtime_error& e) {
            std::println(stderr, "{}", e.what());
        }

        std::println("\nSchur decomposition (A = Q*T*Q^T):");
        std::println("----------------------------------");
        try {
            auto [Q, T_mat] = matrix.Schur();
            std::println("Q (orthogonal):\n{}\n", Q.display(precision));
            std::println("T (upper triangular):\n{}\n", T_mat.display(precision));
            std::println("Checking QTQ^T:\n{}\n", (Q * T_mat * Q.transpose()).display(
                precision));
        } catch (const std::runtime_error& e) {
            std::println(stderr, "Schur: {}", e.what());
        }

        std::println("\nJordan-Chevalley Decomposition (Dunford) (A = D + N, DN = ND):");
        std::println("----------------------------------------------------------------");
        try {
            auto [D, N] = matrix.jordan_chevalley();
            std::println("D:\n{}", D.display(precision));
            std::println("N:\n{}", N.display(precision));

            std::println("\nVerification:");
            std::println("A = D + N:\n{}", (D + N).display(precision));
            std::println("N² = 0:\n{}", (N * N).display(precision));

            auto DN = D * N;
            auto ND = N * D;
            std::println("DN - ND:\n{}", (DN - ND).display(precision));
        } catch (const std::exception& e) {
            std::println(stderr, "Jordan-Chevalley decomposition error: {}", e.what());
        }
    } else {
```

```
        std::unreachable();
    }

    std::println("\nIwasawa Decomposition (A = KDN)");
    std::println("-------------------------------");
    try {
        auto [K, D, N] = matrix.iwasawa();

        std::println("K:");
        std::println("{}", K.display(precision));
        std::println("det(K) = {:.{}f}", K.determinant(), precision);

        std::println("\nD:");
        std::println("{}", D.display(precision));

        std::println("\nN:");
        std::println("{}", N.display(precision));

        std::println("\nVerification A = KDN:");
        Matri2x2<T> KDN = K * D * N;
        std::println("K * D * N:\n{}", KDN.display(precision));

        T error = (matrix - KDN).norm();
        std::println("||A - KDN|| = {:.{}e}", error, precision);

        std::println("\nProperties verification:");
        std::println("K orthogonal? {}", K.is_orthogonal_acces() ? "Yes" : "No");
        D.verifications();
        N.verifications();
        std::println("D diagonal positive? {}", D.is_diagonal_positive_acces() ? "Yes" :
            "No");
        std::println("N upper unipotent? {}", N.is_upper_unipotent_acces() ? "Yes" : "No"
            );
    } catch (const std::runtime_error& e) {
        std::println(stderr, "Iwasawa: {}", e.what());
    }

    std::println("\nPolar Decomposition (A = UP):");
    std::println("----------------------------");

    if (matrix.is_symmetric_acces()) {
        try {
            // Version for symmetric positive definite matrices
            auto [U, P] = matrix.polar_decomposition();
```

```
            std::println("Orthogonal part U:\n{}", U.display(precision));
            std::println("Symmetric positive part P:\n{}", P.display(precision));

            std::println("\nVerification A = UP:");
            std::println("UP:\n{}", (U * P).display(precision));

            std::println("\nVerification P² = A:");
            std::println("P²:\n{}", (P * P).display(precision));

        } catch (const std::runtime_error& e) {
            std::println(stderr, "Polar decomposition (symmetric): {}", e.what());

            std::println("\n=== General Polar Decomposition ===");
            try {
                auto [U, P] = matrix.polar_decomposition_general();
                std::println("Orthogonal part U:\n{}", U.display(precision));
                std::println("Symmetric positive part P:\n{}", P.display(precision));

                std::println("\nVerification A = UP:");
                std::println("UP:\n{}", (U * P).display(precision));
                std::println("U orthogonal? {}",
                    (U.transpose() * U == Matri2x2<T>(T(1), T(0), T(0), T(1))) ? "Yes" : "No");
                std::println("||A - UP|| = {:.{}e}", (matrix - U * P).norm(), precision);

            } catch (const std::runtime_error& e2) {
                std::println(stderr, "General polar: {}", e2.what());
            }
        }
    } else {
        std::println("Matrix not symmetric - using general polar decomposition:");
        try {
            auto [U, P] = matrix.polar_decomposition_general();
            std::println("Orthogonal part U:\n{}", U.display(precision));
            std::println("Symmetric positive part P:\n{}", P.display(precision));

            std::println("\nVerification A = UP:");
            std::println("UP:\n{}", (U * P).display(precision));
            std::println("||A - UP|| = {:.{}e}", (matrix - U * P).norm(), precision);
            std::println("U orthogonal? {}",
                (U.transpose() * U == Matri2x2<T>(T(1), T(0), T(0), T(1))) ? "Yes" : "No");

        } catch (const std::runtime_error& e) {
```

```
            std::println(stderr, "Polar decomposition: {}", e.what());
        }
    }

    std::fflush(stdout);
    std::fflush(stderr);

    matrixPtr = nullptr;

    return EXIT_SUCCESS;
}
```